

Question Bank Management

1. Feature Name

Question Bank Management

2. Purpose

- **Problem Solved:** Quizzes need reusable question content. Authoring questions one by one in the UI is slow and hard to maintain.
 - **Why It Exists:** The feature lets administrators upload complete question banks as JSON files, which are validated, reformatted into relational tables (`Questions`, `Options`), versioned, and made available to the Quiz Engine for building quizzes.
 - **Role in Application:** It is the content source for the Quiz Engine. Quizzes reference `QuestionBankId` and pull their questions from these banks. Admins manage banks; instructors search and reference them when authoring quizzes.
-

3. What the User Can Do

Admin

- Upload a question bank from a JSON file.
- Re-upload a new version of an existing bank.
- Download the stored JSON of a bank.
- Archive / restore a bank.
- Delete a bank (blocked if quizzes reference it).
- Search banks by title, subject, or tag.

Instructor

- Search and view question banks (Admin-only write actions are not exposed to instructors).
-

4. Feature Workflow

[Admin uploads a bank]

- POST `/api/v1/questionbanks/upload` (multipart file, Admin, 6 MB request limit)
- `QuestionBankService.UploadAsync`
- `JsonValidationService.Validate(fileName, fileSize, jsonContent)`
 - .json extension
 - ≤ 5 MB
 - parses as JSON
 - Title/Subject/Questions required
 - per-question: text, ≥ 2 options, correct answer present + matches an option, valid difficulty, explanation, no in-file duplicates
- If errors → 400 with per-question validation messages, nothing saved
- Deserialize `QuestionBankUploadModel`
- Versioning: if a bank with same Title+Subject exists, version = max+1
- QuestionBank row created (StoredJson kept verbatim, WordCount = questions count)
- ParseQuestions: builds Question + Option rows (IsCorrect flagged)
- Save + audit log `QUESTION_BANK_UPLOADED`
- Returns `QuestionBankUploadResult`

5. How It Works Internally

Validation Pipeline

`JsonValidationService.Validate()` runs sequential checks that short-circuit on fatal errors:

1. **Extension** — must be `.json`; otherwise errors returned immediately.
2. **File size** — must be ≤ 5 MB (`MaxFileSize = 5 * 1024 * 1024`); otherwise immediate return.
3. **JSON parsing** — deserialized with `PropertyNameCaseInsensitive = true`; a `JsonException` is captured and returned as `"Invalid JSON format."`
4. **Required root fields** — non-empty `Title`, `Subject`, and a non-empty `Questions` array.
5. **Per-question checks** — non-empty question text; ≥ 2 options; non-empty `CorrectAnswer`; `CorrectAnswer` must equal one of the options (case-insensitive `OrdinalIgnoreCase`); difficulty must be `easy` / `medium` / `hard`; explanation present; duplicate question text detection via a `HashSet<string>` (`StringComparer.OrdinalIgnoreCase`).

Errors are `ValidationError` objects with `QuestionIndex` (1-based), `Reason`, and optional `Detail`. The controller formats them as `Question N: <reason> (<detail>)`.

Versioning

In `UploadAsync`:

- Queries `QuestionBanks` for the highest `Version` where `Title == model.Title && Subject == model.Subject`.

- New bank version = that version + 1 (or 1 if none exists).
- `UploadVersionAsync(id, ...)` creates a new row for an existing bank with `Version = existing.Version + 1`, copying `Title`, `Subject`, and `Tags`.

Each version is a **separate row** (full snapshot), so history is preserved. `StoredJson` keeps the original file content verbatim for download.

Question Parsing (JSON → relational)

`QuestionBankService.ParseQuestions()`:

- `Difficulty` string (`easy / medium / hard`, case-insensitive) is mapped to the `Difficulty` enum; unknown values default to `Easy`.
- For each question, the correct option is flagged by comparing each option text with `CorrectAnswer` using `StringComparison.OrdinalIgnoreCase`.
- `Option.DisplayOrder` = its zero-based index in the JSON array.
- Question text and explanation are trimmed.

Search

`SearchAsync(title, subject, tag)` builds a filtered EF query (`Title.Contains`, `Subject.Contains`, `Tags.Contains`) ordered by `CreatedAt` descending. The frontend wrapper returns `totalCount = items.length` — the backend does not paginate.

Download

`GetStoredJsonAsync(id)` returns the `StoredJson` string; the controller serves it as `application/json` with filename `questionbank-{id}.json`.

Status Lifecycle

`QuestionBankStatus`: `Draft` (on upload/version) → `Active` (after restore) → `Archived` (on archive). `ArchiveAsync` sets `ArchivedAt`; `RestoreAsync` only works when status is `Archived` (else `InvalidOperationException("Question Bank is already active.")`).

Delete Guard

`DeleteAsync` loads `qb.Quizzes` and blocks deletion with an error listing the quiz count if any quiz references the bank.

6. Frontend Implementation

Layer	File Path	Responsibility
Page	<code>frontend/src/pages/admin/AdminQuestionBanksPage.tsx</code>	Upload, version, download, archive/restore/delete, search UI (with JSON template download and validation hints)
Service	<code>frontend/src/services/questionBankService.ts</code>	upload, search, getById, download, uploadVersion, archive, restore, delete
Types	<code>frontend/src/types/questionBank.types.ts</code>	Bank DTO + status types

7. Backend Implementation

Layer	File Path	Responsibility
Controller	<code>backend/Controllers/QuestionBankController.cs</code>	Upload, search, get, download, version, delete, archive, restore endpoints
Service	<code>backend/Services/QuestionBank/QuestionBankService.cs</code>	Validation orchestration, versioning, parsing, lifecycle
Service	<code>backend/Services/Validation/JsonValidationService.cs</code>	Strict JSON schema/content validation
DTO	<code>backend/DTOs/QuestionBank/*</code>	<code>QuestionBankUploadModel</code> , <code>QuestionBankUploadResult</code> , <code>QuestionBankResponseDto</code>
Interface	<code>backend/Interfaces/Services/IQuestionBankService.cs</code>	Service contract
Interface	<code>backend/Interfaces/Services/IValidationService.cs</code>	Validation contract

8. Database Implementation

Entity/Table	Purpose	Important Relationship
QuestionBanks	Bank metadata + raw stored JSON	1→N Questions, 1→N Quizzes; has Title, Subject, Tags, Version, StoredJson, Status
Questions	Parsed evaluation questions	FK QuestionBankId → QuestionBanks (Cascade); 1→N Options
Options	Answer options per question	FK QuestionId → Questions (Cascade); IsCorrect flag, DisplayOrder

- The full original JSON is persisted in `QuestionBanks.StoredJson`, enabling re-download.
- Each uploaded version creates a new `QuestionBanks` row (history preserved).
- Questions/options are cascade-deleted when their bank/version row is removed.

9. Security & Authorization

- Upload, version, download, delete, archive, restore: `[Authorize(Roles = "Admin")]`.
- Search and get-by-id: `[Authorize(Roles = "Admin,Instructor")]`.
- Upload endpoints are size-limited at the HTTP level (`[RequestSizeLimit(6 * 1024 * 1024)]`).
- Server-side validation prevents malformed or oversized files from being stored.
- No secrets are involved; all validation is content-based.

10. Important Business Rules

1. **File constraints:** only `.json`, ≤ 5 MB, valid JSON.
2. **Required fields:** `Title`, `Subject`, non-empty `Questions` array.
3. **Per-question:** ≥ 2 options; the correct answer must exactly match one option (case-insensitive); valid `easy|medium|hard` difficulty; explanation required; no duplicate question text within the file.
4. **On any validation error, nothing is saved** ("No data was saved").
5. **Versioning:** same Title+Subject → new row with incremented version; otherwise version 1.
6. **Lifecycle:** uploaded banks start as `Draft`; only `Archived` banks may be restored.
7. **Delete blocked** while quizzes reference the bank.

8. **Difficulty mapping:** unknown/excluded difficulties default to `Easy`.

11. Example of Internal Execution

Step 1: Valid upload

- **User Action:** Admin selects `math-bank.json` (5 questions, correct format).
- **Frontend:** `questionBankService.upload` posts multipart to `POST /api/v1/questionbanks/upload`.
- **Controller:** size check (under 6 MB), then `QuestionBankService.UploadAsync`.
- **Validation:** extension `.json` ✓, size ✓, `JsonSerializer` parses ✓, root fields present ✓, per-question checks pass ✓ (no errors).
- **Version:** no bank titled "Math Bank" exists → `version = 1`.
- **DB:** `QuestionBanks` row (StoredJson = original file) + 5 `Questions` + their `Options` (correct one flagged via case-insensitive compare) are inserted.
- **Response:** `QuestionBankUploadResult { Success = true, Message = "Question Bank uploaded successfully (v1)." }`
- **Frontend UI:** the bank appears in the list with version 1.

Step 2: Invalid upload rejected

- Admin uploads a file with a question missing options.
- Validation returns `Question 3: Question must have at least 2 options`.
- Controller returns 400 with the per-question error list and `"Validation failed. No data was saved."`; no DB write occurs.

Step 3: Version upload

- Admin re-uploads the same Title+Subject.
 - `existing.Version = 1` → new row `version = 2`; search now shows two rows (v1, v2).
-

12. Files Involved

Frontend

- `frontend/src/pages/admin/AdminQuestionBanksPage.tsx`
- `frontend/src/services/questionBankService.ts`
- `frontend/src/types/questionBank.types.ts`

Backend

- `backend/Controllers/QuestionBankController.cs`
- `backend/Services/QuestionBank/QuestionBankService.cs`
- `backend/Services/Validation/JsonValidationService.cs`
- `backend/DTOs/QuestionBank/*`
- `backend/Interfaces/Services/IQuestionBankService.cs`
- `backend/Interfaces/Services/IValidationService.cs`

Database

- `backend/Entities/QuestionBank.cs`
 - `backend/Entities/Question.cs`
 - `backend/Entities/Option.cs`
 - `backend/Entities/Difficulty.cs`
 - `backend/Entities/QuestionBankStatus.cs`
 - `backend/Configurations/QuestionBankConfiguration.cs` (and related EF configurations)
-

13. Functionality

- Upload question banks as JSON files
- Validate JSON schema, file type, file size, and per-question content
- Auto-version banks with matching title/subject
- Store original JSON and re-download it
- Parse questions/options into relational tables
- Search banks by title, subject, or tag
- Archive / restore banks
- Delete banks (with quiz-reference guard)
- Per-question validation error reporting (question index + reason)